

Informe técnico — Pipeline de Datos (20GB) y Entrenamiento IA

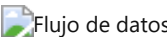
Este documento describe, de forma imprimible y accionable, cómo el proyecto RaceTelemetryAI ingiere ~20GB de datos de telemetría, los transforma en conjuntos de entrenamiento, entrena modelos, y los usa en inferencia en tiempo real. Incluye flujos, esquemas relacionales y prácticas de gobernanza.

Resumen Ejecutivo

- Fuentes: sesiones/vueltas/frames de telemetría (velocidad, rpm, freno, dirección, temperaturas, combustible), anotaciones de sectores y eventos.
- Ingesta: archivos locales en `DataFiles/` (CSV/JSON/Parquet) procesados incrementalmente por `DataManager` .
- ETL: validación, limpieza, normalización por pista, enriquecimiento (sectorización por path del circuito, features compuestas), partición por sesiones.
- Entrenamiento: datasets por pista y global, con objetivos como riesgos por sector, ventana de pit, alertas mecánicas.
- Inferencia: `riskEngine` y `geminIService` combinan métricas, reglas y análisis IA para recomendaciones.

Estructura de Datos (20GB)

- Almacenamiento local: `DataFiles/` (no versionado), carpeta por pista y temporada.
- Tipos:
 - `Session` : metadatos (trackId, vehículo, piloto, fecha, condiciones).
 - `Lap` : tiempos, velocidad media, combustible usado.
 - `TelemetryFrame` : series temporales de `speed` , `rpm` , `gear` , `throttle` , `brake` , `steer` , `fuelLevel` , `oilTemp` , `waterTemp` , `tireTemps[]` , `gps` .
 - `Sector` : índices y tiempos por vuelta.
 - `AlertEvent` , `Recommendation` : generadas por análisis.



Esquema Relacional (ER)

El proyecto usa un esquema lógico compatible con CSV/Parquet. Las relaciones clave son:



Entidades principales:

- `Session` 1..N `Lap` , 1..N `TelemetryFrame` , 1..N `AlertEvent` , 1..N `Recommendation` .
- `TrackProfile` define sectores, geometría (path SVG) y coeficientes.

Pipeline ETL y Entrenamiento



Ingesta

- `DataManager.parseDataFiles(trackName)` lee CSV/Parquet por bloques (chunk size configurable) para evitar OOM.
- Validación: esquema esperado, rangos físicos (rpm, temp), y coherencia de tiempos.
- Indexado: `sessionId` , `lapId` , `timestamp` .

Limpieza y Normalización

- Interpolación de huecos cortos, resample a frecuencia uniforme por pista.
- Normalización por pista: `speedMax` , `rpmMax` , `brakeMax` , `steerMax` desde `trackModels.json` .
- Alineación espacial: uso de `trackPaths.json` y `mapTransform` para proyectar progreso sobre el centro de pista (0..100).

Enriquecimiento y Feature Engineering

- Derivadas y suavizados: `dSpeed/dt` , `dBrake/dt` , variabilidad de `steer` .
- Cálculo de carga térmica: integrales aproximadas de `oilTemp` , `waterTemp` por sector.
- Consumo de combustible: diferencia por vuelta con compensación de pit.
- Riesgos compuestos por sector: combinación de freno, temperatura, desgaste (si disponible), y márgenes de rpm.
- Eventos: detección de outliers, micro-bloqueos de frenos, overheating.

Particionado

- Por sesión y pista para evitar fuga de datos: train/val/test ~70/15/15.
- Estratificación por condiciones (seco/lluvia) si las etiquetas existen.

Modelos y Objetivos

- `SectorRisk` (0..1): regresión/estimador compuesto.
- `Pit Window` : clasificación binaria/temporal (abrir/cerrar ventana estimada).
- `Alerts` : clasificación (overheat, brake-fade, fuel-critical).

Entrenamiento

- Pipelines reproducibles con configuración por pista.
- Métricas: MAE/MSE para riesgos, F1/AUC para alertas, tasa de falsos positivos.
- Registro de artefactos: versión de dataset, hiperparámetros y métricas por pista.

Inferencia y Uso en la App

- `riskEngine.ts` : aplica calibración (`calibration.json`), normalización (`trackModels.json`) y reglas; produce `SectorRisk` y análisis visual.
- `geminiservice.ts` : genera recomendaciones textuales a partir de métricas agregadas.
- `CircuitViewer.tsx` : muestra progreso sobre el `d` del circuito (0..100); la calibración alinea path con el PDF.

Gobernanza, Trazabilidad y Rendimiento

- Carpeta `DataFiles/` y `TrackMap/` como solo lectura en desarrollo.
- Versionado de configuraciones (`trackModels.json` , `calibration.json` , `trackPaths.json`).
- Logs de ETL y métricas de validación.
- Procesamiento incremental y por bloques para 20GB.

Anexos

- Rutas clave de configuración:
 - `/DataFiles/config/calibration.json` — `mapTransform` por pista.
 - `/DataFiles/models/trackModels.json` — normalizaciones y sectores.
 - `/DataFiles/config/trackPaths.json` — `d` SVG del centro de pista.
- Ejemplos de features:
 - `brake_intensity = moving_avg(brake, 500ms)`
 - `temp_rise_rate = d(oilTemp)/dt` por sector
 - `steer_variability = std(steer)` por segmento

Para imprimir: este archivo enlaza a PNG generados desde Mermaid. Puedes abrirlo en el navegador o convertir a PDF con las instrucciones que acompañan.